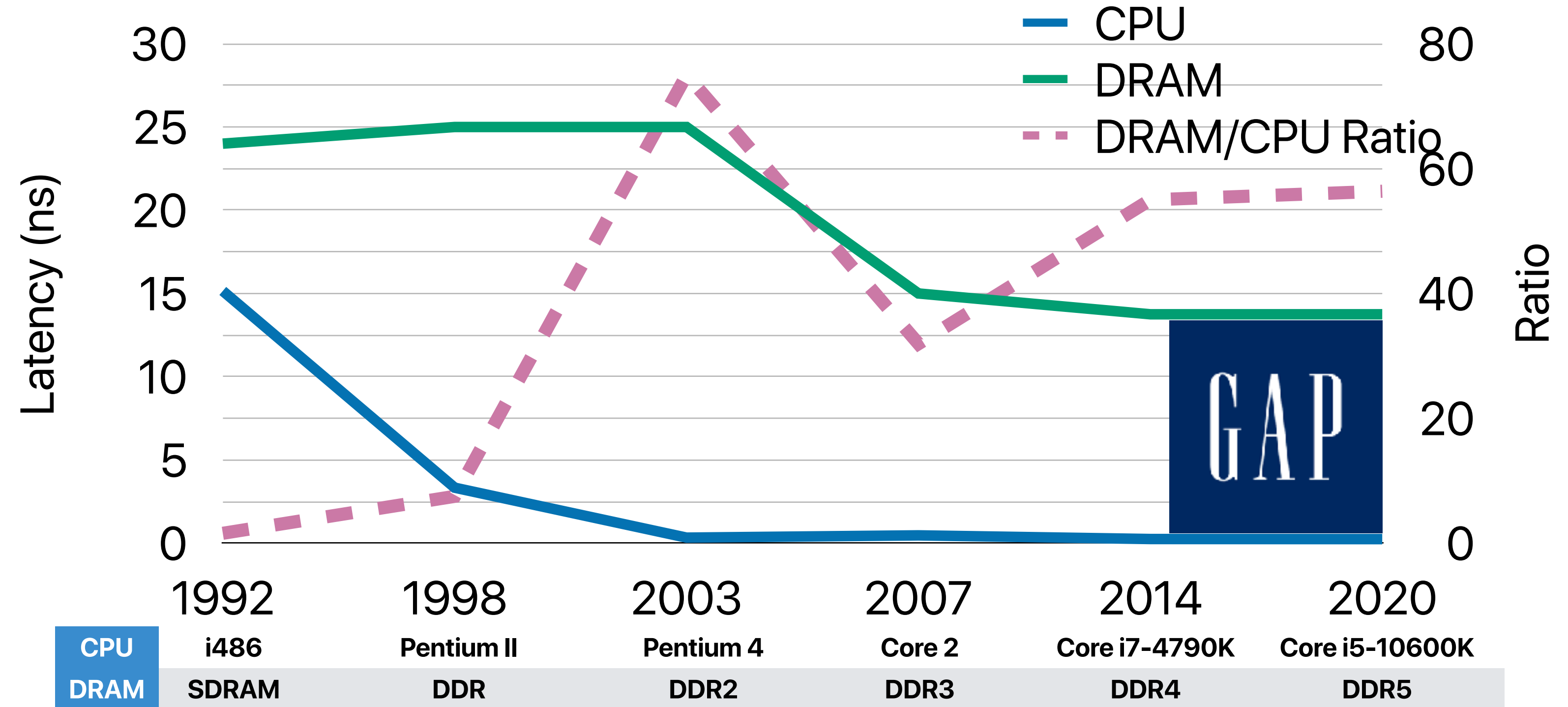


Memory Hierarchy (Preview)

Outline

- The memory-all problem
- The “predictable” code behavior
- Designing a cache that captures the predictability

The "latency" gap between CPU and DRAM



20% is under-estimating ...

Instruction class	MIPS examples	HLL correspondence	Frequency	
			Integer	Ft. pt.
Arithmetic	add, sub, addi	Operations in assignment statements	16%	48%
Data transfer	lw, sw, lb, lbu, lh, lhu, sb, lui	References to data structures, such as arrays	35%	36%
Logical	and, or, nor, andi, ori, sll, srl	Operations in assignment statements	12%	4%
Conditional branch	beq, bne, slt, slti, sltiu	If statements and loops	34%	8%
Jump	j, jr, jal	Procedure calls, returns, and case/switch statements	2%	0%

FIGURE 2.48 MIPS instruction classes, examples, correspondence to high-level program language constructs, and percentage of MIPS instructions executed by category for the average integer and floating point SPEC CPU2006 benchmarks.

Figure 3.24 in Chapter 3 shows average percentage of the individual MIPS instructions executed.

Recap: Speedup and Amdahl's Law?

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$ $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 1 — each optimization has an upper bound

- **Corollary 2 — make the common case (the most time consuming case) fast!**

- Corollary 3: Optimization has a moving target
- Corollary 4: Exploiting more parallelism from a program is the key to performance gain in modern architectures
- Corollary 5: Single-core performance still matters

$$\begin{aligned} Speedup_{max}(f_1, \infty) &= \frac{1}{(1-f_1)} \\ Speedup_{max}(f_2, \infty) &= \frac{1}{(1-f_2)} \\ Speedup_{max}(f_3, \infty) &= \frac{1}{(1-f_3)} \\ Speedup_{max}(f_4, \infty) &= \frac{1}{(1-f_4)} \end{aligned}$$

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

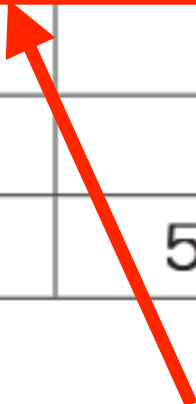
$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!

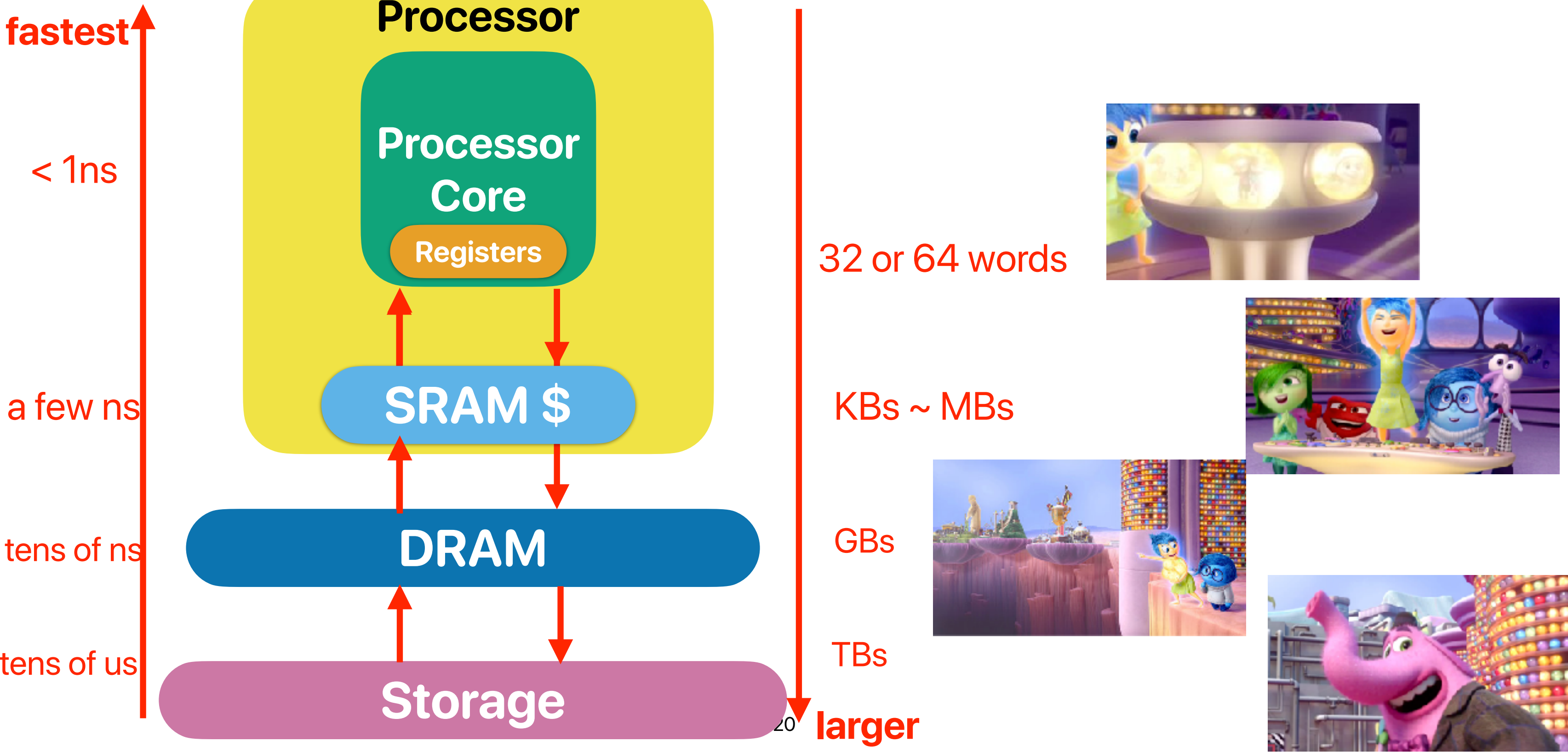
Alternatives?

Memory technology	Typical access time	\$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$10–\$20
Flash semiconductor memory	5,000–50,000 ns	\$0.75–\$1.00
Magnetic disk	5,000,000–20,000,000 ns	\$0.05–\$0.10



Fast, but expensive \$\$\$

Memory Hierarchy



L1? L2? L3?

CPU-Z - ID : vfljgr

CPU Mainboard Memory SPD Graphics Bench About

Processor

Name	AMD Ryzen 7 7700X		
Code Name	Raphael	Max TDP	105 W
Package	Socket AM5 (LGA1718)		
Technology	5 nm	Core Voltage	1.288 V



Specification

AMD Ryzen 7 7700X 8-Core Processor			
Family	F	Model	1
Ext. Family	19	Ext. Model	61
Stepping	2	Revision	RPL-B2

Instructions

MMX(+), SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, SSE4A, x86-64, AMD-V, AES, AVX, AVX2, AVX512, FMA3, SHA

Clocks (Core #0)

Core Speed	5188.99 MHz
Multiplier	x 52.0 (4 - 55.5)
Bus Speed	99.79 MHz
Rated FSB	

Cache

L1 Data	8 x 32 KB
L1 Inst.	8 x 32 KB
Level 2	8 x 1024 KB
Level 3	32 MBytes

Selection Socket #1 Cores 8 Threads 16

CPU-Z Ver. 2.09.0.x64 Tools Validate Close

CPU-Z - ID : pk15b

CPU Mainboard Memory SPD Graphics Bench About

Processor

Name	Intel Core i7 14700K		
Code Name	Raptor Lake	Max TDP	125 W
Package	Socket 1700 LGA		
Technology	10 nm	Core Voltage	1.412 V



Specification

Intel(R) Core(TM) i7-14700K			
Family	6	Model	7
Ext. Family	6	Ext. Model	B7
Stepping	1	Revision	B0

Instructions

MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, EM64T, VT-x, AES, AVX, AVX2, FMA3, SHA

Clocks (Core #0)

Core Speed	5287.07 MHz
Multiplier	x 53.0 (8 - 55)
Bus Speed	99.76 MHz
Rated FSB	

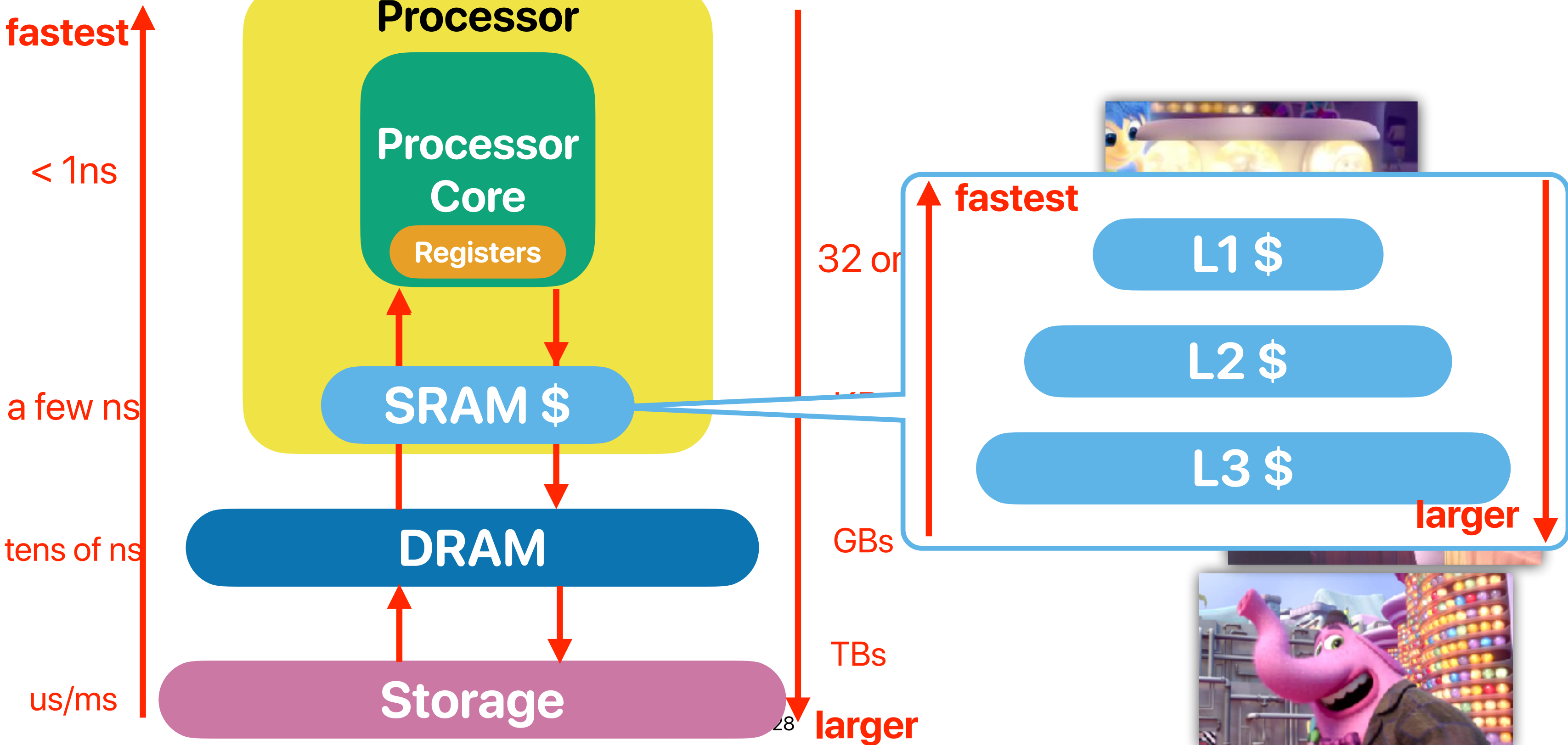
Cache

L1 Data	8 x 48 KB + 12 x 32 KB
L1 Inst.	8 x 32 KB + 12 x 64 KB
Level 2	8 x 2 MB + 3 x 4 MB
Level 3	33 MBytes

Selection Socket #1 Cores 8 + 12 Threads 28

CPU-Z Ver. 2.08.0.x64 Tools Validate Close

Memory Hierarchy



L1? L2? L3?

Can we really “predict” upcoming data accurately (e.g., 90%) with such small caches?

CPU-Z - ID : vfljgr	
CPU Mainboard Memory SPD Graphics Bench About	
Processor	
Name	AMD Ryzen 7 7700X
Code Name	Raphael
Max TDP	105 W
Package	AM5
Technology	5 nm
Core Voltage	1.2 V
Specification	
AMD Ryzen 7 7700X 8-Core Processor	
Ext. Family	19
Ext. Model	1
Revision	RPL12
Instructions	
MMX(+), SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, SSE4A, x86-64, AMD-V, AES, AVX, AVX2, AVX512, FMA3, SHA	
Clocks (Core #0)	
Core Speed	5188.99 MHz
Multiplier	x 52.0 (4 - 55.5)
Bus Speed	99.79 MHz
Rated FSB	
Cache	
L1 Data	8 x 32 KB
L1 Inst.	8 x 32 KB
Level 2	8 x 1024 KB
Level 3	32 MBytes
Selection Socket #1	
Cores	8
Threads	16
CPU-Z Ver. 2.09.0.x64 Tools Validate Close	

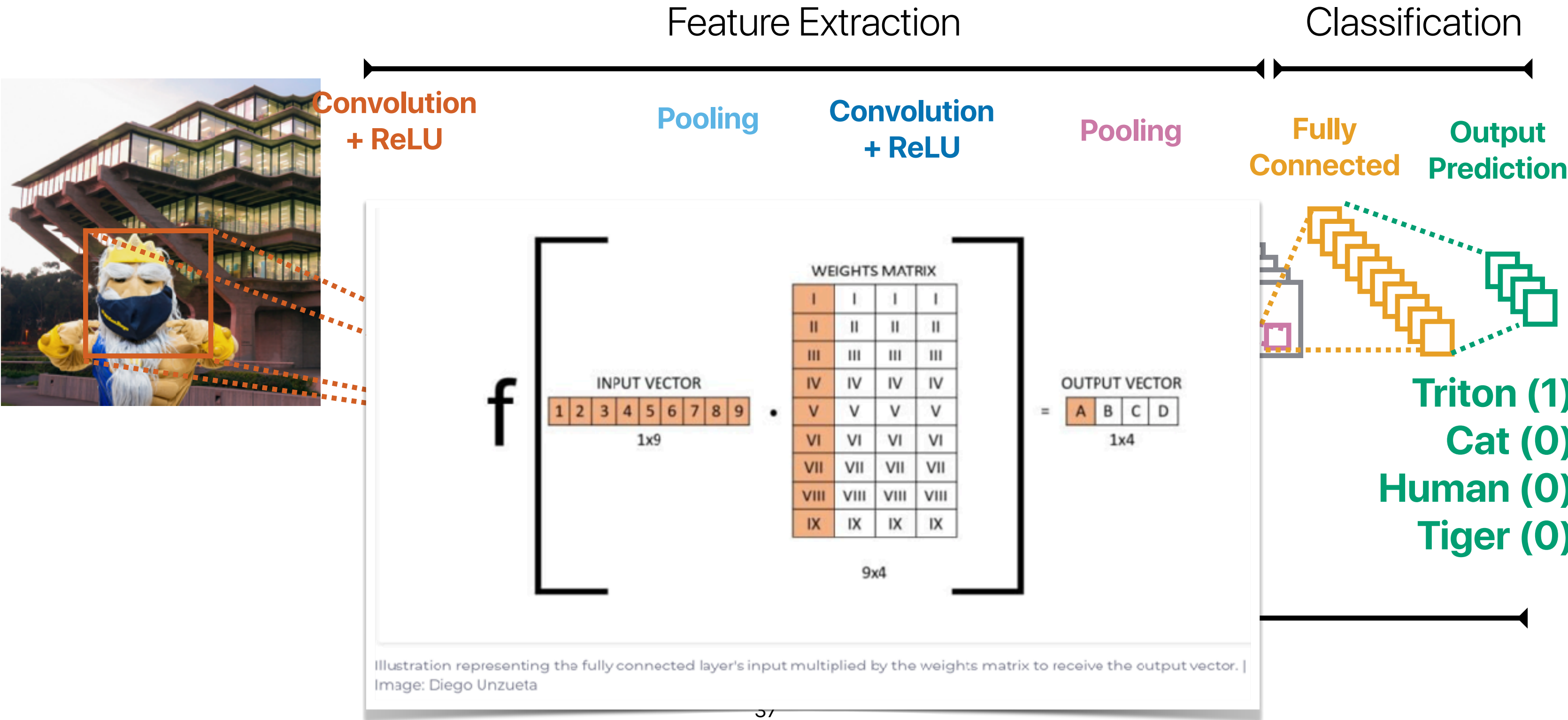
CPU-Z - ID : pk15b	
CPU Mainboard Memory SPD Graphics Bench About	
Processor	
Name	Intel Core i7 14700K
Code Name	Raptor Lake
Max TDP	125 W
Package	LGA1700
Technology	14 nm
Core Voltage	1.1 V
Specification	
Intel(R) Core(TM) i7-14700K	
Ext. Family	6
Ext. Model	b7
Revision	B0
Instructions	
MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, EM64T, VT-x, AES, AVX, AVX2, FMA3, SHA	
Clocks (Core #0)	
Core Speed	5287.07 MHz
Multiplier	x 53.0 (8 - 55)
Bus Speed	99.76 MHz
Rated FSB	
Cache	
L1 Data	8 x 48 KB + 12 x 32 KB
L1 Inst.	8 x 32 KB + 12 x 64 KB
Level 2	8 x 2 MB + 3 x 4 MB
Level 3	33 MBytes
Selection Socket #1	
Cores	8 + 12
Threads	28
CPU-Z Ver. 2.08.0.x64 Tools Validate Close	

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions

The predictability of your code

The Machine Learning Inference Pipeline



Code also has locality

keep going to the
next instruction —
spatial locality

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

**repeat many times —
temporal locality!**

```
i = 0;  
while(i < m) {  
    result = 0;  
    j = 0;  
    while(j < n) {  
        a = matrix[i][j];  
        b = vector[j];  
        temp = a*b;  
        result = result + temp;  
    }  
    output[i] = result;  
    i++;  
}
```

Locality

- Spatial locality — application tends to visit nearby stuffs in the memory

- Code — the current instruction, and then the next instruction

Most of time, your program is just visiting a very small amount of data/instructions within a given window

- Code — loops, frequently invoked functions

- Typically tens of static instructions — at most several KBs

- Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions
- Caching is possible! Most of time, we only work on a small amount of data!
 - Spatial locality
 - Temporal locality

Designing a hardware to exploit locality

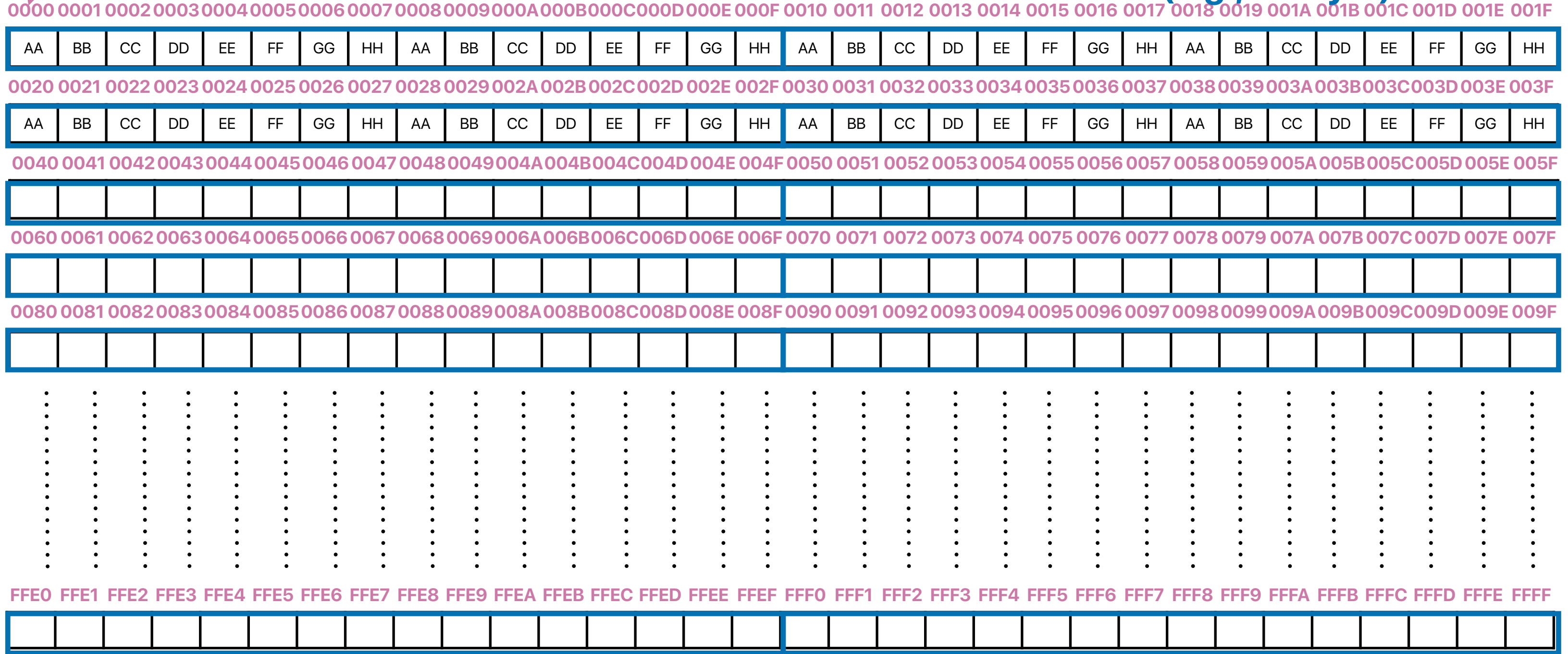
- Spatial locality — application tends to visit nearby stuffs in the memory

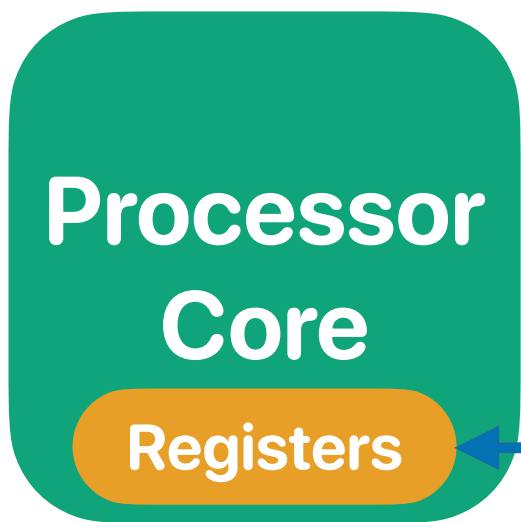
We need to "cache consecutive memory locations" every time — the cache should store a "block" of code/data

- Temporal locality — application revisit the same thing again and again
 - Code — loops, frequently invoked functions
 - Typically tens of static instructions — at most several KBs
 - Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)

Block and the memory space

Each byte of memory location has an "address" Partition the space with fixed-size "blocks" (e.g., 16-byte)



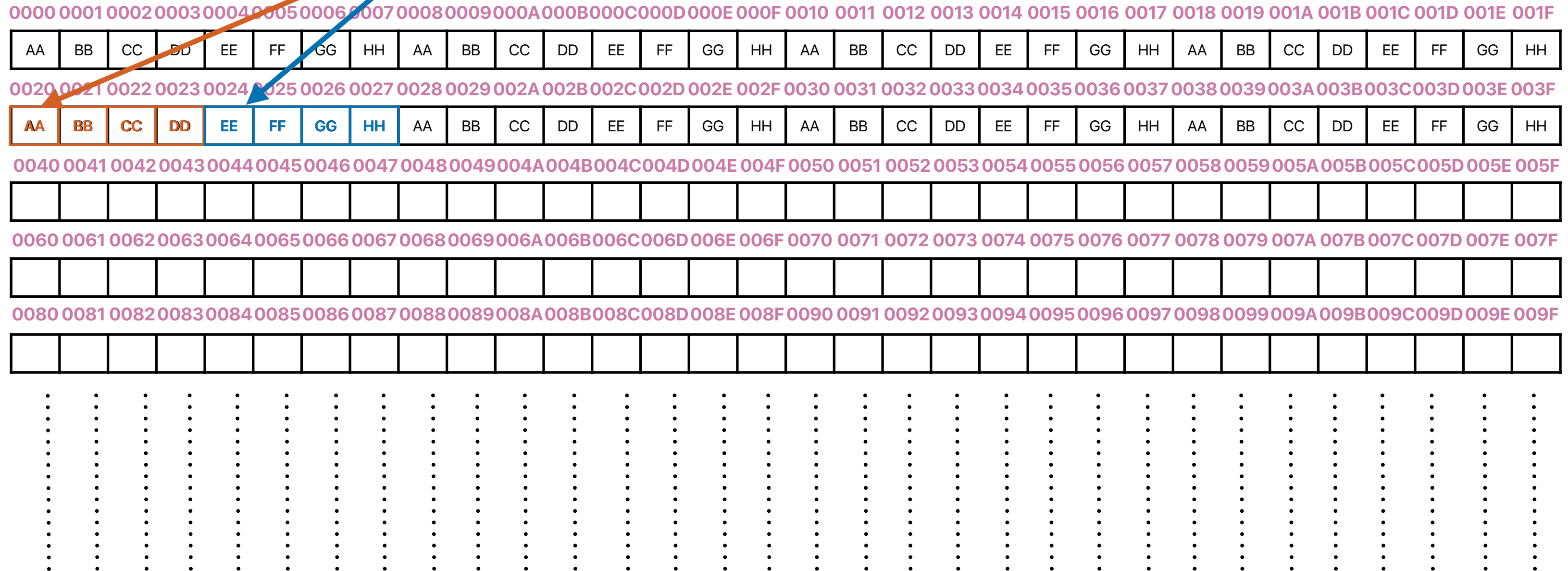


When there is a "movl"

movl (0x0024), %eax

movl (0x0020), %eax

Every "movl" has to visit the slow memory!



Recap: Locality

- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

**Simply caching one block
isn't enough**

Designing a hardware to exploit locality

- Spatial locality — application tends to visit nearby stuffs in the memory

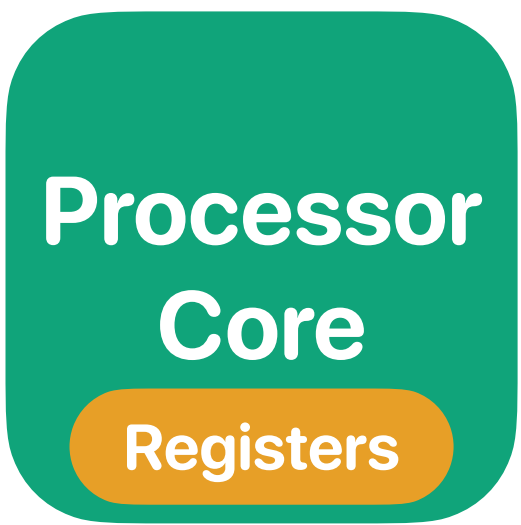
We need to “cache consecutive memory locations” every time — the cache should store a “block” of code/data

- Temporal locality — application revisit the same thing again and again

We need to “cache frequently used memory blocks”

— the cache should store a few blocks several KBs

— the cache must be able to distinguish blocks • Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)



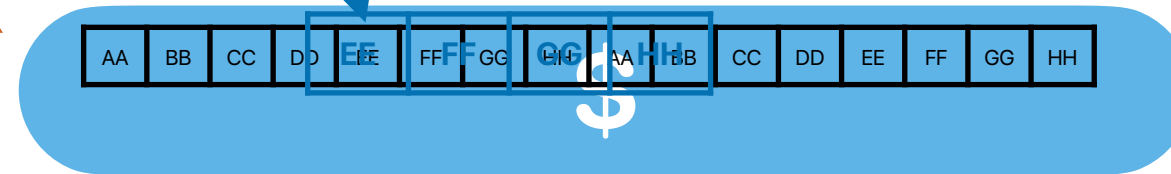
How to tell who is there?

?															
0123456789ABCDEF															
?															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															

Registers

Let's cache a "block"!

```
movl (0x0024), %eax
```

~~movl (0x0020), %eax~~

0000 0001 0002 0003 0004 0005 0006 0007 0008 0009 000A 000B 000C 000D 000E 000F 0010 0011 0012 0013 0014 0015 0016 0017 0018 0019 001A 001B 001C 001D 001E 001F

AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

0020 0021 0022 0023 0024 0025 0026 0027 0028 0029 002A 002B 002C 002D 002E 002F 0030 0031 0032 0033 0034 0035 0036 0037 0038 0039 003A 003B 003C 003D 003E 003F

AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH	AA	BB	CC	DD	EE	FF	GG	HH
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

0040 0041 0042 0043 0044 0045 0046 0047 0048 0049 004A 004B 004C 004D 004E 004F 0050 0051 0052 0053 0054 0055 0056 0057 0058 0059 005A 005B 005C 005D 005E 005F

[illegible]

0060 0061 0062 0063 0064 0065 0066 0067 0068 0069 006A 006B 006C 006D 006E 006F 0070 0071 0072 0073 0074 0075 0076 0077 0078 0079 007A 007B 007C 007D 007E 007F

[illegible]

0080 0081 0082 0083 0084 0085 0086 0087 0088 0089 008A 008B 008C 008D 008E 008F 0090 0091 0092 0093 0094 0095 0096 0097 0098 0099 009A 009B 009C 009D 009E 009F

**the address in each block starts
with the same "prefix"**



The diagram shows a green rounded square representing the 'Processor Core'. Inside the square, the text 'Processor Core' is written in white. At the bottom of the square, there is an orange rounded rectangle containing the text 'Registers' in white.

Registers

How to tell who is there?

the common address prefix in each block

tag array

0x000	This is CS 203:
0x001	Advanced Compute
0xF07	r Architecture!
0x100	This is CS 203:
0x310	Advanced Compute
0x450	r Architecture!
0x006	This is CS 203:
0x537	Advanced Compute
0x266	r Architecture!
0x307	This is CS 203:
0x265	Advanced Compute
0x80A	r Architecture!
0x620	This is CS 203:
0x630	Advanced Compute
0x705	r Architecture!
0x216	This is CS 203:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0x0000	0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	0x0009	0x000A	0x000B	0x000C	0x000D	0x000E	0x000F
01	2	3	4	5	6	7	8	9	A	B	C	D	E	F	



How to tell w

block offset

tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

Tell if the block here can be used

Tell if the block here is modified

		Valid Bit	Dirty Bit	tag	data
					0123456789ABCDEF
1	1			0x000	This is CSE13:
1	1			0x001	Advanced Computer Architecture!
1	0			0xF07	
0	1			0x100	This is CS 203:
1	1			0x310	Advanced Computer Architecture!
1	1			0x450	
0	1			0x006	This is CS 203:
0	1			0x537	Advanced Computer Architecture!
1	1			0x266	
1	1			0x307	This is CS 203:
0	1			0x265	Advanced Computer Architecture!
0	1			0x80A	
1	1			0x620	This is CS 203:
1	1			0x630	Advanced Computer Architecture!
1	0			0x705	
0	1			0x216	This is CS 203:

Blocksize == Linesize

```
[5]: # Your CS203 Cluster
! cs203 demo "lscpu | grep 'Model name'; getconf -a | grep CACHE"

ssh htseng@horsea " srun -N1 -p datahub lscpu | grep 'Model name'"
Model name:                  12th Gen Intel(R) Core(TM) i3-12100F
ssh htseng@horsea " srun -N1 -p datahub getconf -a | grep CACHE"
LEVEL1_ICACHE_SIZE           32768
LEVEL1_ICACHE_ASSOC           8
LEVEL1_ICACHE_LINESIZE       64
LEVEL1_DCACHE_SIZE           49152
LEVEL1_DCACHE_ASSOC           12
LEVEL1_DCACHE_LINESIZE       64
LEVEL2_CACHE_SIZE             1310720
LEVEL2_CACHE_ASSOC            10
LEVEL2_CACHE_LINESIZE        64
LEVEL3_CACHE_SIZE             12582912
LEVEL3_CACHE_ASSOC            12
LEVEL3_CACHE_LINESIZE        64
LEVEL4_CACHE_SIZE             0
LEVEL4_CACHE_ASSOC            0
LEVEL4_CACHE_LINESIZE         0
```

Take-aways: inside out our memory hierarchy

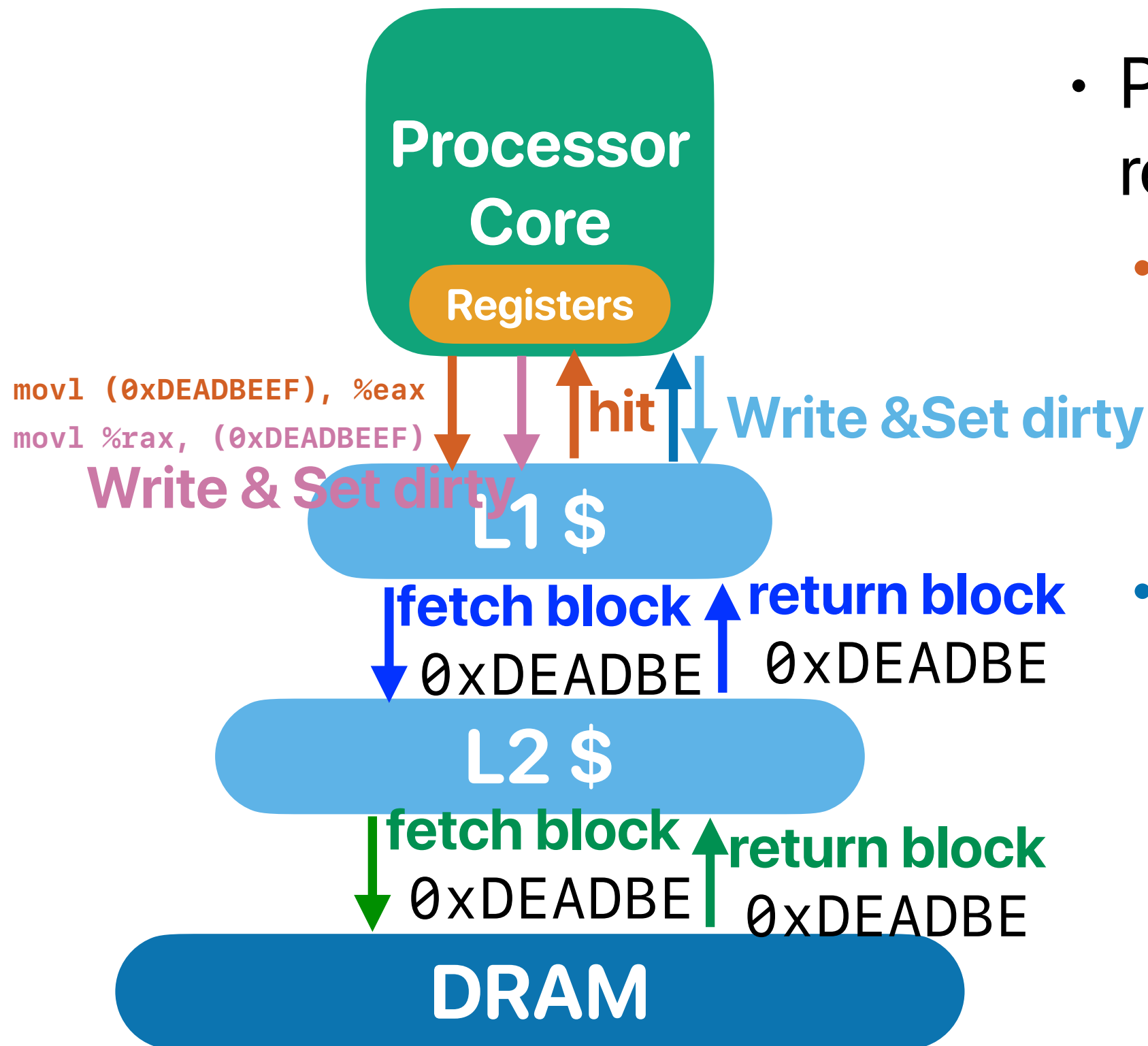
- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions
- Caching is possible! Most of time, we only work on a small amount of data!
 - Spatial locality
 - Temporal locality
- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks

**Put everything all together:
How cache interacts with CPU**

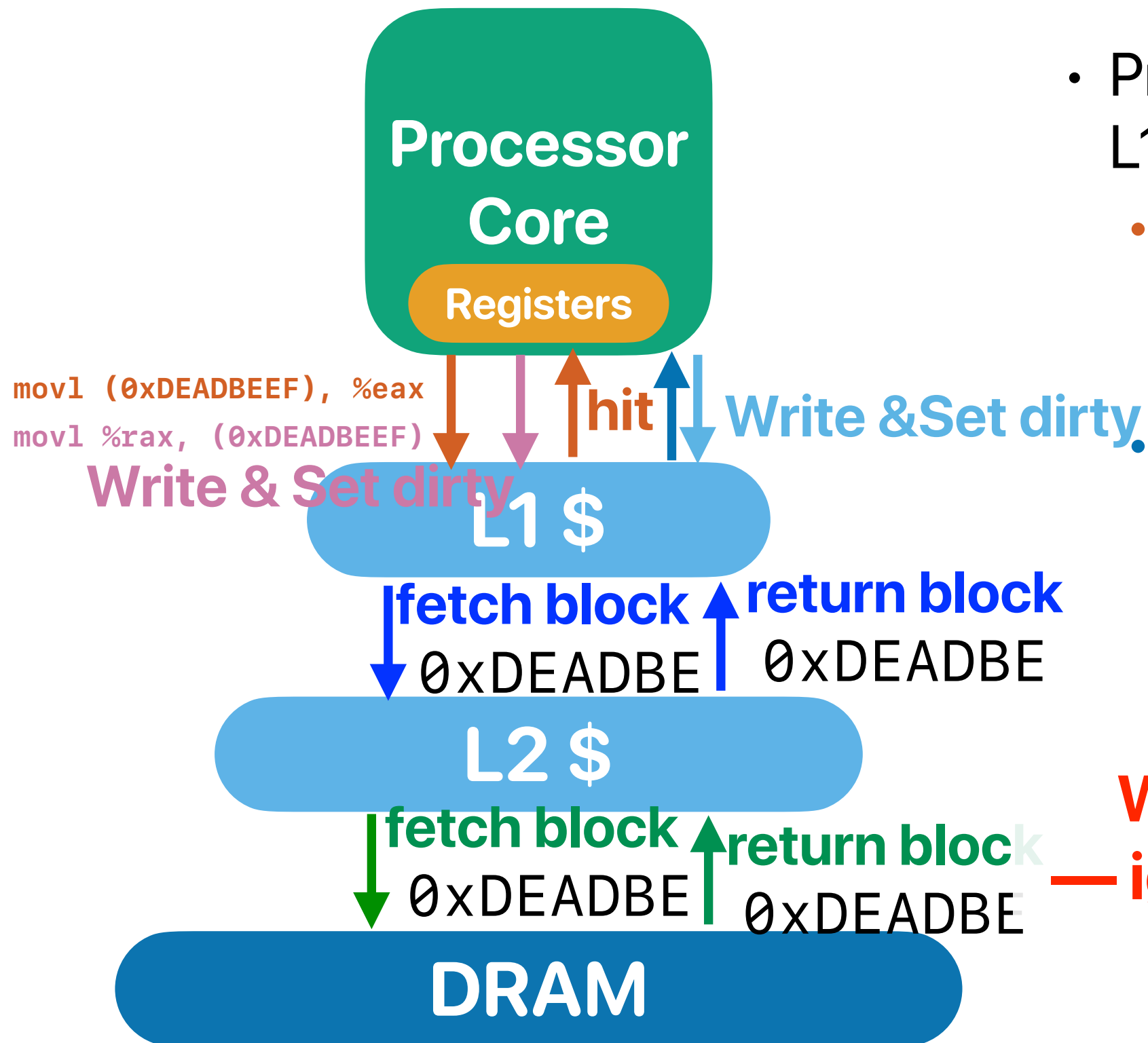
Processor/cache interaction



- Processor sends memory access request to L1-\$
 - if hit & it's a read
 - Read: return data
 - Write: Update "ONLY" in L1 and set DIRTY
 - Why don't we write to L2?
 - Too slow
- if miss
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - Present the write "ONLY" in L1 and set DIRTY

What if we run out of \$ blocks?

Processor/cache interaction

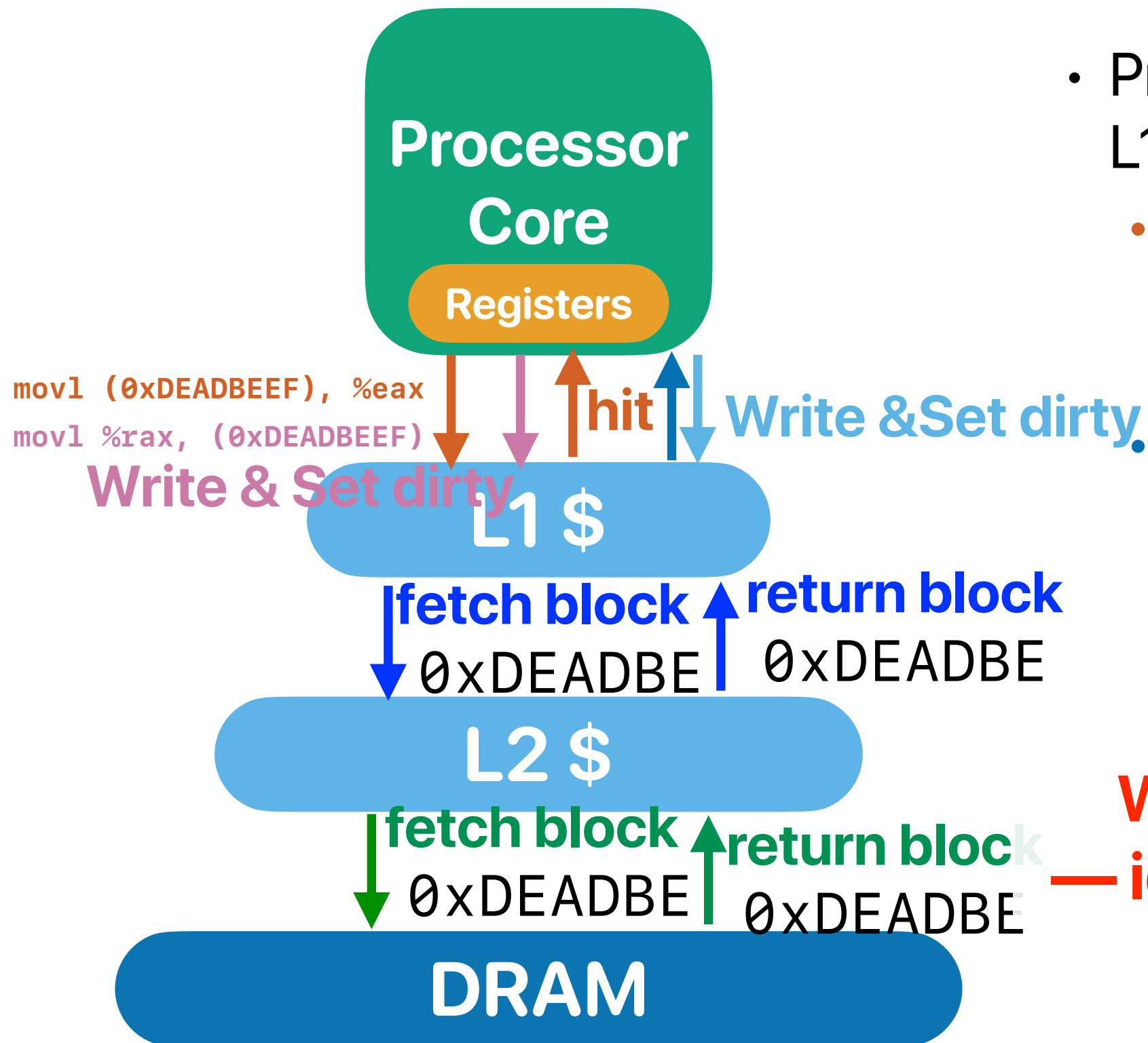


- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - Read: return data
 - Write: Update "ONLY" in L1 and set DIRTY
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- Present the write "ONLY" in L1 and set DIRTY

Processor/cache interaction

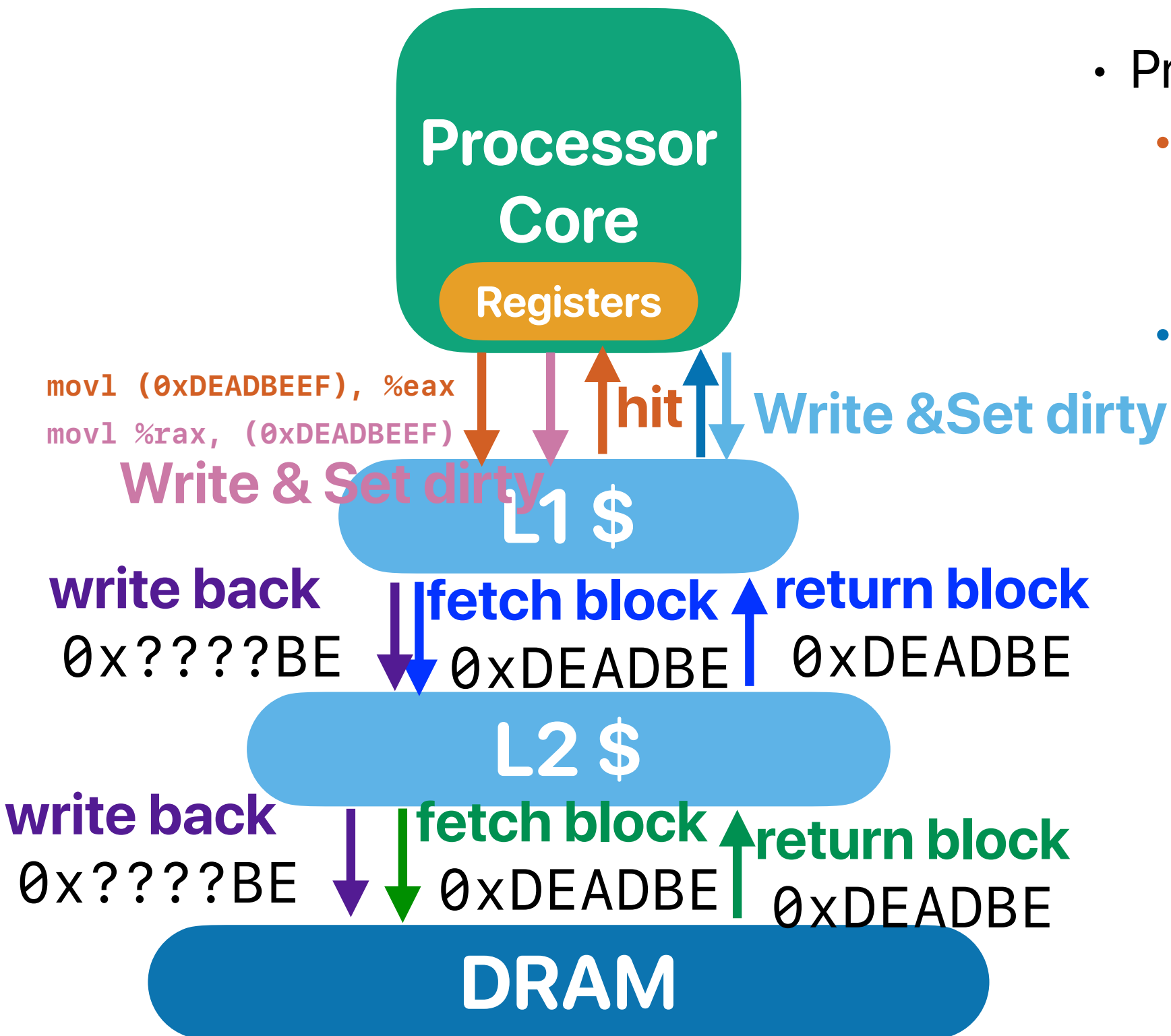


- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - Read: return data
 - Write: Update "ONLY" in L1 and set DIRTY
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- Present the write "ONLY" in L1 and set DIRTY

Processor/cache interaction

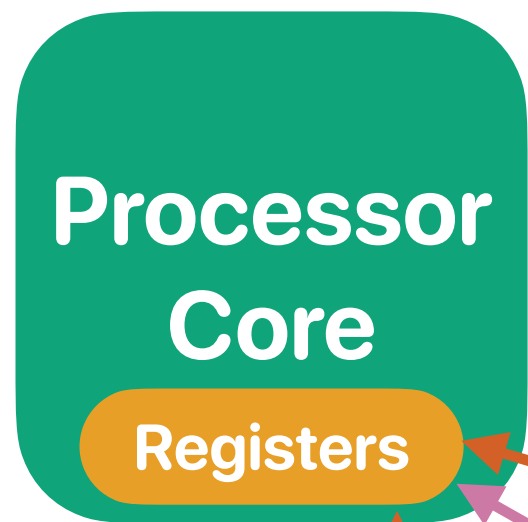


- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read: return data**
 - **Write: Update "ONLY" in L1 and set DIRTY**
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy
 - If the victim block is "dirty" & "valid"
 - **Write back** the block to lower-level memory hierarchy
 - If write-back or fetching causes any miss, repeat the same process
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use

How to tell w



block offset

tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

The complexity of search the matching tag—
 $O(n)$ —will be slow if our cache size grows!

Can we search things faster?
—hash table! $O(1)$

Tell if the block here can be used

Tell if the block here is modified

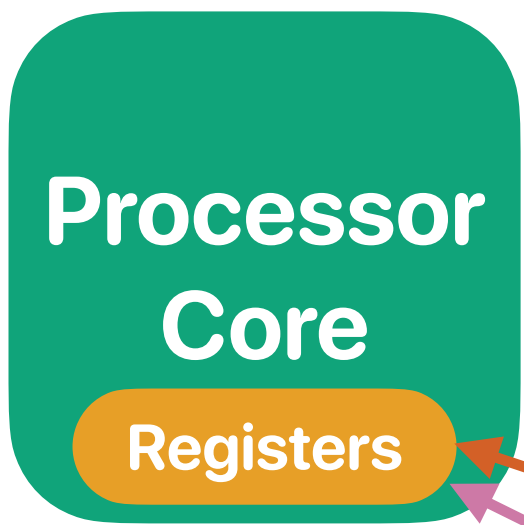
Valid Bit

Dirty Bit

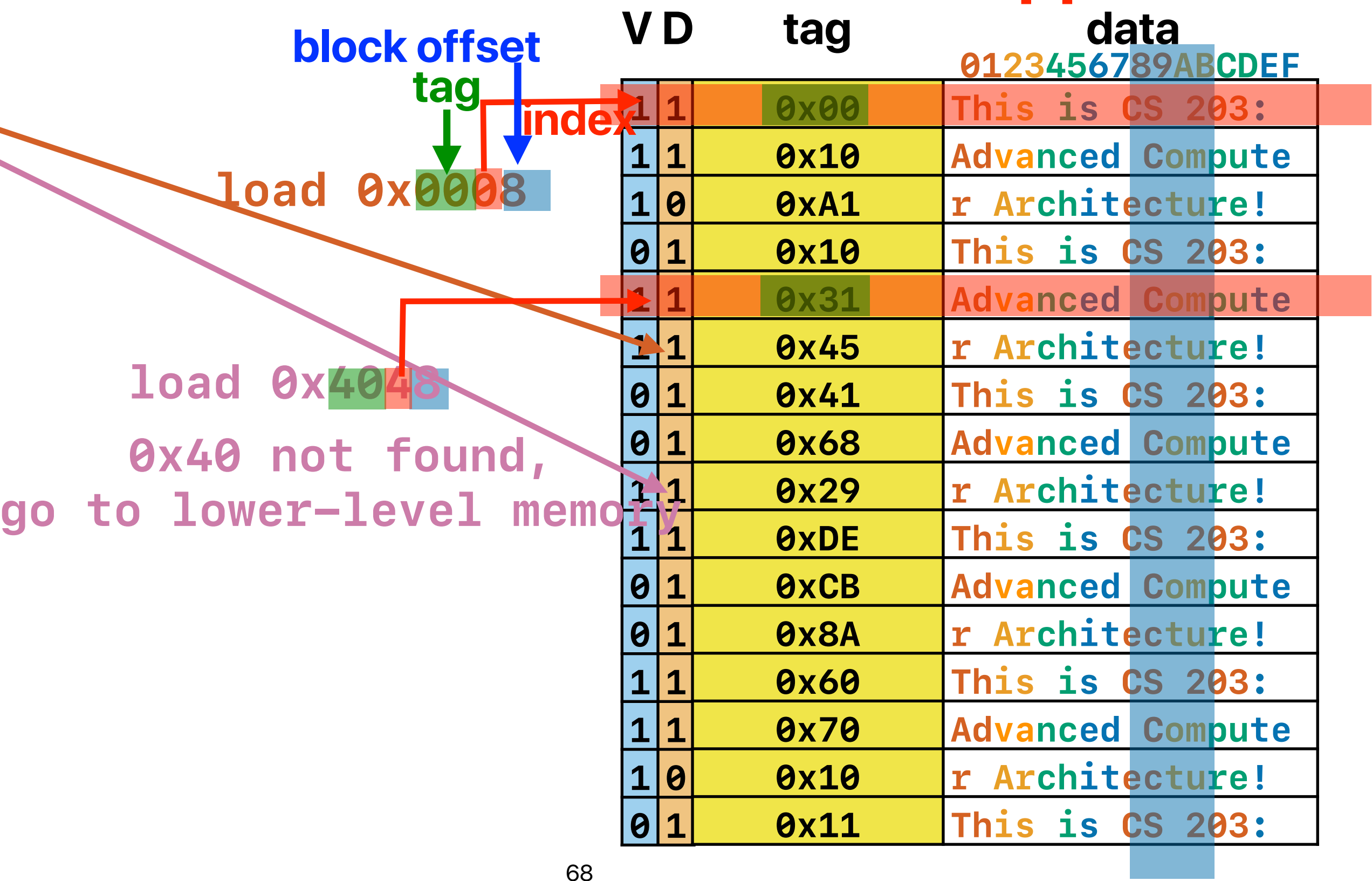
tag

data

				0123456789ABCDEF
1	1	0x000	This is CSE13:	
1	1	0x001	Advanced Computer	
1	0	0xF07	r Architecture!	
0	1	0x100	This is CS 203:	
1	1	0x310	Advanced Computer	
1	1	0x450	r Architecture!	
0	1	0x006	This is CS 203:	
0	1	0x537	Advanced Computer	
1	1	0x266	r Architecture!	
1	1	0x307	This is CS 203:	
0	1	0x265	Advanced Computer	
0	1	0x80A	r Architecture!	
1	1	0x620	This is CS 203:	
1	1	0x630	Advanced Computer	
1	0	0x705	r Architecture!	
0	1	0x216	This is CS 203:	

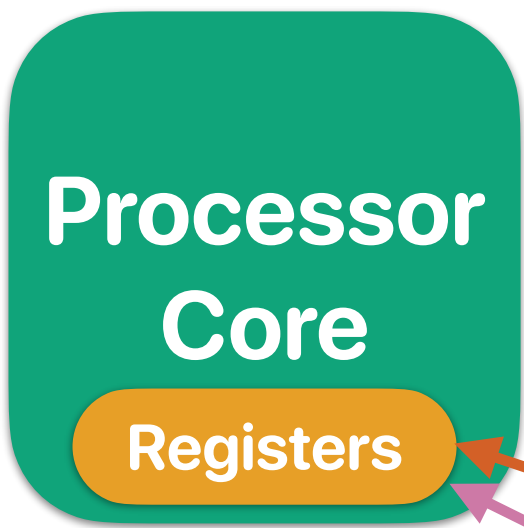


Hash-like structure — direct-mapped cache

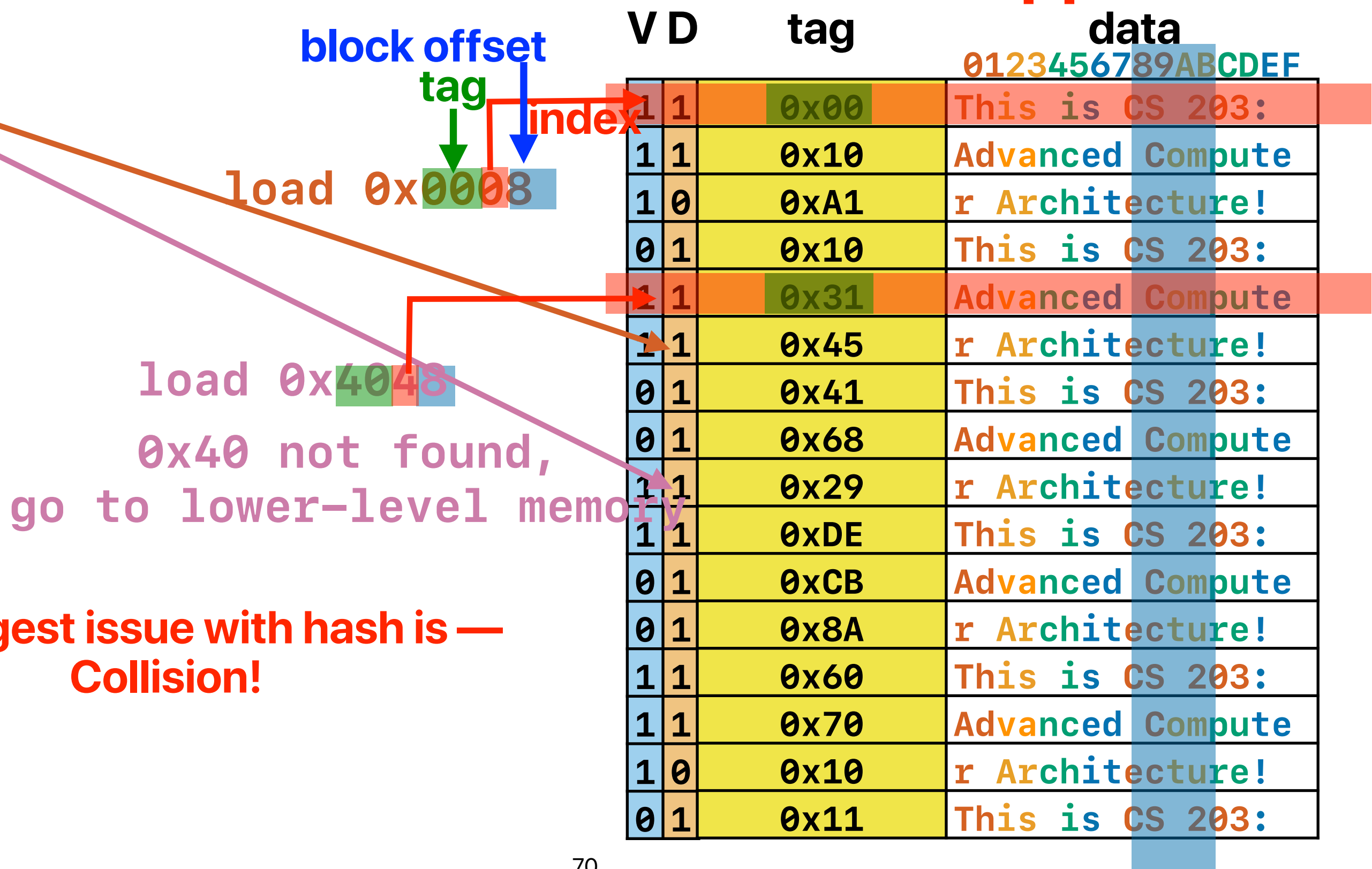


Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time



Hash-like structure — direct-mapped cache

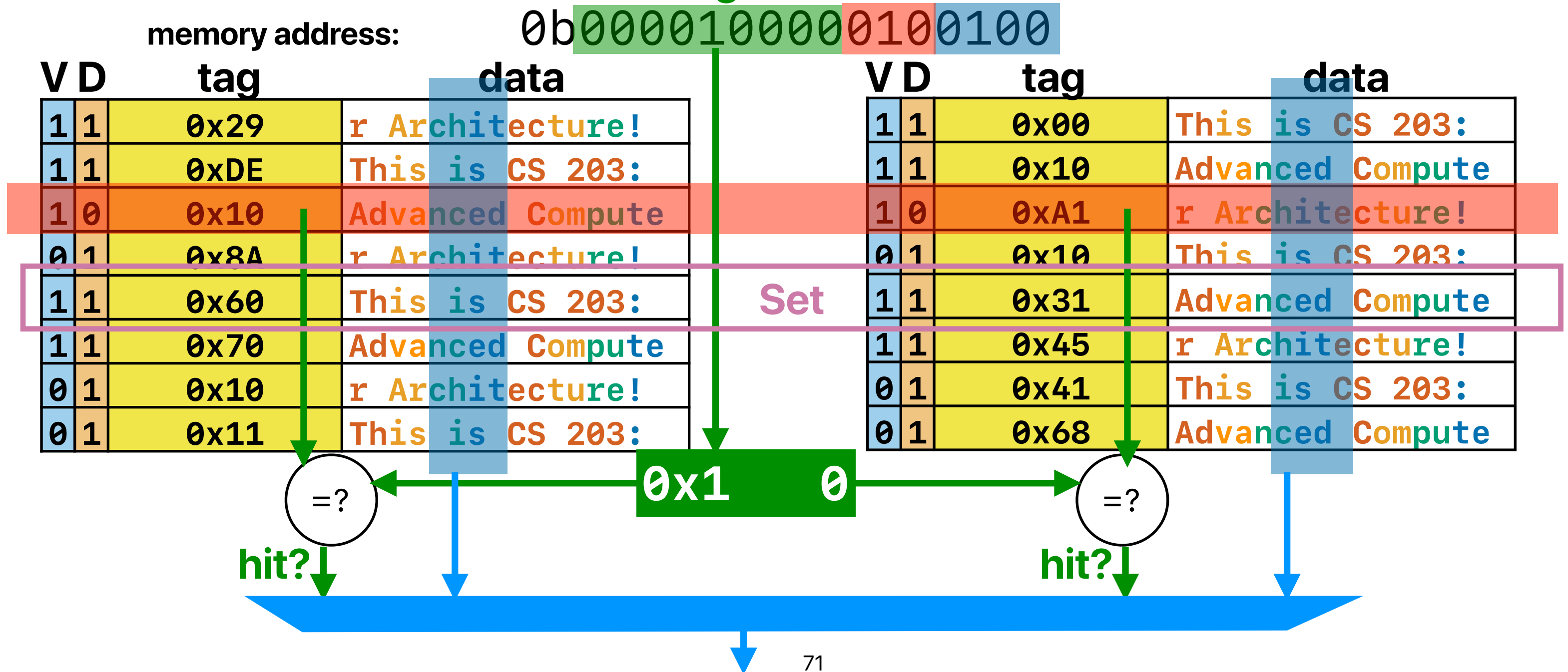


The biggest issue with hash is — Collision!

Way-associative cache

memory address: 0x0 8 2 4

tag set index block offset



Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem

Way-associative cache

memory address: $0x0$ 8 2 4

set block

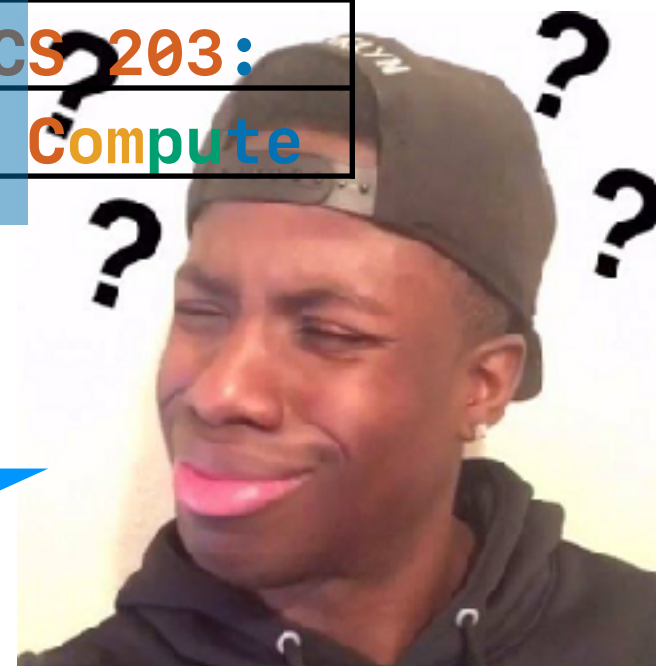
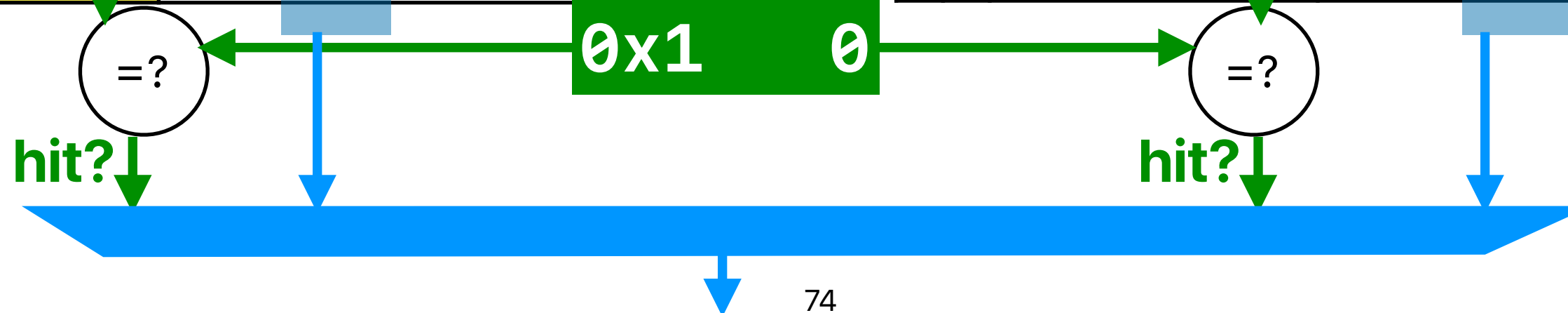
tag index offset

memory address: $0b00001000000100100$

V	D	tag	data
1	1	$0x29$	r Architecture!
1	1	$0xDE$	This is CS 203:
1	0	$0x10$	Advanced Compute
0	1	$0x8A$	r Architecture!
1	1	$0x60$	This is CS 203:
1	1	$0x70$	Advanced Compute
0	1	$0x10$	r Architecture!
0	1	$0x11$	This is CS 203:

V	D	tag	data
1	1	$0x00$	This is CS 203:
1	1	$0x10$	Advanced Compute
1	0	$0xA1$	r Architecture!
0	1	$0x10$	This is CS 203:
1	1	$0x31$	Advanced Compute
1	1	$0x45$	r Architecture!
0	1	$0x41$	This is CS 203:
0	1	$0x68$	Advanced Compute

Set



The A, B, Cs of your cache

$$C = ABS$$

- **C: Capacity** in data arrays
- **A: Way-Associativity** — how many blocks within a set
 - N-way: N blocks in a set, $A = N$
 - 1 for direct-mapped cache
- **B: Block Size (Linesize)**
 - How many bytes in a block
- **S: Number of Sets:**
 - A set contains blocks sharing the same index
 - 1 for fully associate cache



Corollary of $C = ABS$

memory address: 0b 000010000 010 0100

tag set index block offset

- number of bits in **block** offset — $\lg(\mathbf{B})$
- number of bits in **set** index: $\lg(\mathbf{S})$
- tag bits: $\text{address_length} - \lg(\mathbf{S}) - \lg(\mathbf{B})$
 - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- $(\text{address} / \text{block_size}) \% \mathbf{S} = \text{set index}$

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem
- $C = A B S$
 - C: capacity
 - A: Associativity
 - S: Number of sets
 - $\lg(S)$: Number of bits in set index
 - $\lg(B)$: Number of bits in block offset

Simulate the cache!

Simulate a direct-mapped cache

- A direct mapped (1-way) cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- $\lg(16) = 4$: 4 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits
- For example: 0x 8 0 0 0 0 0 8 0

= 0b1000 0000 0000 0000 0000 0000 0000 1000 0000



Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

Matrix vector revisited

for(uint64_t i = 0; i < m; i++) {
 result = 0;
 for(uint64_t j = 0; j < n; j++) {
 result += matrix[i][j]*vector[j];
 }
 output[i] = result;
}

tagindex

	Address (Hex)	Address (Binary)
&a[0][0]	0x558FE0A1D330	0b10101011000111111100000101000011101001100110000
&b[0]	0x558FE0A1DC30	0b10101011000111111100000101000011101110000110000
&a[0][1]	0x558FE0A1D338	0b10101011000111111100000101000011101001100111000
&b[1]	0x558FE0A1DC38	0b101010110001111111000001010000111011100000111000
&a[0][2]	0x558FE0A1D340	0b10101011000111111100000101000011101001101000000
&b[2]	0x558FE0A1DC40	0b101010110001111111000001010000111011100001000000
&a[0][3]	0x558FE0A1D348	0b10101011000111111100000101000011101001101001000
&b[3]	0x558FE0A1DC48	0b101010110001111111000001010000111011100001001000
&a[0][4]	0x558FE0A1D350	0b10101011000111111100000101000011101001101010000
&b[4]	0x558FE0A1DC50	0b101010110001111111000001010000111011100001010000
&a[0][5]	0x558FE0A1D358	0b10101011000111111100000101000011101001101011000
&b[5]	0x558FE0A1DC58	0b101010110001111111000001010000111011100001011000
&a[0][6]	0x558FE0A1D360	0b10101011000111111100000101000011101001101100000
&b[6]	0x558FE0A1DC60	0b101010110001111111000001010000111011100001100000
&a[0][7]	0x558FE0A1D368	0b10101011000111111100000101000011101001101101000
&b[7]	0x558FE0A1DC68	0b101010110001111111000001010000111011100001101000
&a[0][8]	0x558FE0A1D370	0b10101011000111111100000101000011101001101110000
&b[8]	0x558FE0A1DC70	0b101010110001111111000001010000111011100001110000
&a[0][9]	0x558FE0A1D378	0b10101011000111111100000101000011101001101111000
&b[9]	0x558FE0A1DC78	0b101010110001111111000001010000111011100001111000

Simulate a direct-mapped cache

tag index

	V	D	Tag	Data
0	0	0		
1	0	0		
2	0	0		
3	1	0	0x558FE0A1DC	b[0], b[1]
4	1	0	0x558FE0A1DC	b[2], b[3]
5	0	0		
6	0	0		
7	0	0		
8	0	0		
9	0	0		
10	0	0		
11	0	0		
12	0	0		
13	0	0		
14	0	0		
15	0	0		

This cache doesn't work!!!
— collisions!

Address (Hex)		
&a[0][0]	0x558FE0A1D330	miss
&b[0]	0x558FE0A1DC30	miss
&a[0][1]	0x558FE0A1D338	miss
&b[1]	0x558FE0A1DC38	miss
&a[0][2]	0x558FE0A1D340	miss
&b[2]	0x558FE0A1DC40	miss
&a[0][3]	0x558FE0A1D348	miss
&b[3]	0x558FE0A1DC48	miss
&a[0][4]	0x558FE0A1D350	miss
&b[4]	0x558FE0A1DC50	miss
&a[0][5]	0x558FE0A1D358	miss
&b[5]	0x558FE0A1DC58	miss
&a[0][6]	0x558FE0A1D360	miss
&b[6]	0x558FE0A1DC60	miss
&a[0][7]	0x558FE0A1D368	miss
&b[7]	0x558FE0A1DC68	miss
&a[0][8]	0x558FE0A1D370	miss
&b[8]	0x558FE0A1DC70	miss
&a[0][9]	0x558FE0A1D378	
&b[9]	0x558FE0A1DC78	

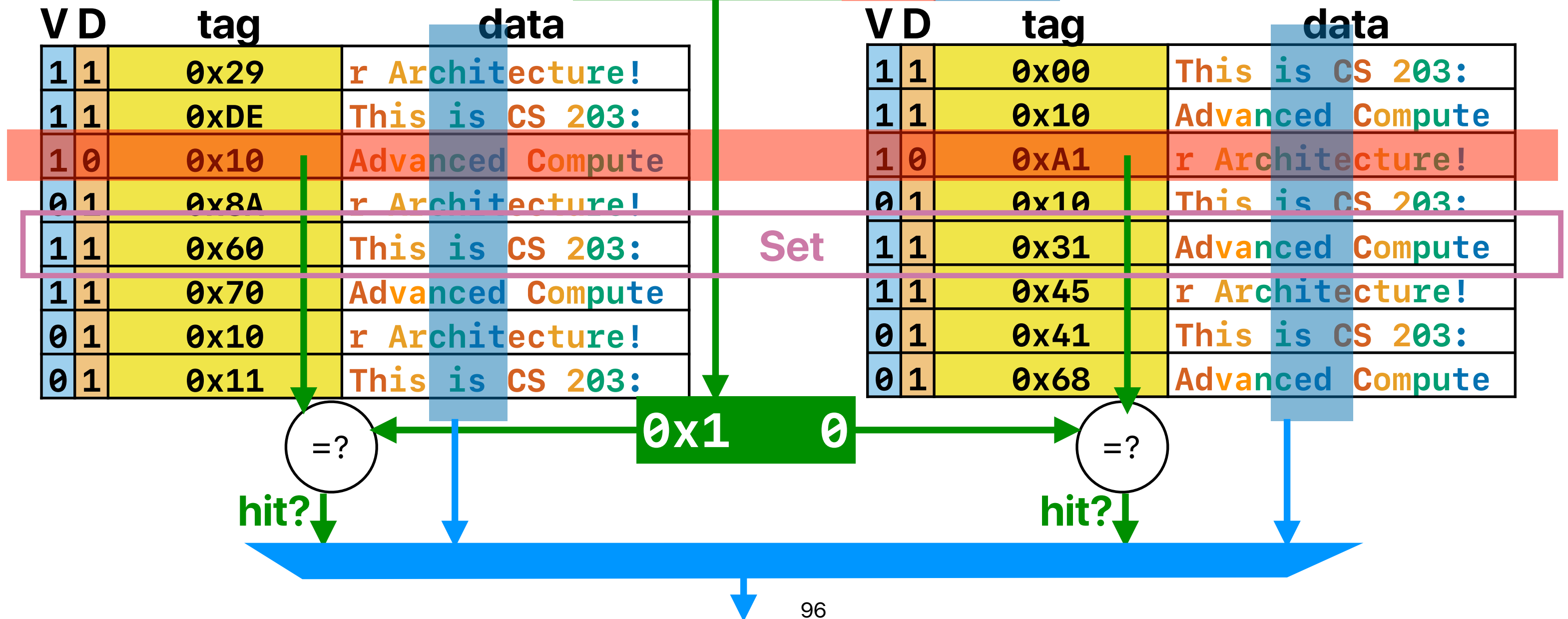
Way-associative cache

memory address: $0x0$ 8 2 4

set block

tag index offset

memory address: $0b00001000000100100$



Now, 2-way, same-sized cache

- A 2-way cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$

- # of sets = $\frac{16}{2} = 8$ (2-way: 2 blocks in a set)

- $\lg(8) = 3$: 3 bits are used for the index

- $\lg(16) = 4$: 4 bits are used for the byte offset

- The tag is $64 - (4 + 4) = 56$ bits

- For example: $0x \quad 8 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 8 \quad 0$
= $0b \quad 1000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 1000 \quad 0000$

tag

index

offset

Matrix vector revisited

	tag index		tag index	
	tag		index	
	Address (Hex)		Address (Binary)	
for(uint64_t i = 0; i < m; i++) {	&a[0][0]	0x558FE0A1D330	0b10101011000111111100000101000011101001100110000	
result = 0;	&b[0]	0x558FE0A1DC30	0b10101011000111111100000101000011101110000110000	
for(uint64_t j = 0; j < n; j++) {	&a[0][1]	0x558FE0A1D338	0b10101011000111111100000101000011101001100111000	
result += matrix[i][j]*vector[j];	&b[1]	0x558FE0A1DC38	0b10101011000111111100000101000011101110000111000	
}	&a[0][2]	0x558FE0A1D340	0b10101011000111111100000101000011101001101000000	
output[i] = result;	&b[2]	0x558FE0A1DC40	0b10101011000111111100000101000011101110001000000	
}	&a[0][3]	0x558FE0A1D348	0b10101011000111111100000101000011101001101001000	
	&b[3]	0x558FE0A1DC48	0b10101011000111111100000101000011101110001001000	
	&a[0][4]	0x558FE0A1D350	0b10101011000111111100000101000011101001101010000	
	&b[4]	0x558FE0A1DC50	0b10101011000111111100000101000011101110001010000	
	&a[0][5]	0x558FE0A1D358	0b10101011000111111100000101000011101001101011000	
	&b[5]	0x558FE0A1DC58	0b10101011000111111100000101000011101110001011000	
	&a[0][6]	0x558FE0A1D360	0b10101011000111111100000101000011101001101100000	
	&b[6]	0x558FE0A1DC60	0b10101011000111111100000101000011101110001100000	
	&a[0][7]	0x558FE0A1D368	0b10101011000111111100000101000011101001101101000	
	&b[7]	0x558FE0A1DC68	0b10101011000111111100000101000011101110001101000	
	&a[0][8]	0x558FE0A1D370	0b10101011000111111100000101000011101001101110000	
	&b[8]	0x558FE0A1DC70	0b10101011000111111100000101000011101110001110000	
	&a[0][9]	0x558FE0A1D378	0b10101011000111111100000101000011101001101111000	
	&b[9]	0x558FE0A1DC78	0b10101011000111111100000101000011101110001111000	

Simulate a 2-way cache

V	D	Tag	Data	V	D	Tag	Data
0	0			0	0		
0	0			0	0		
0	0			0	0		
1	0	0xAB1FC143A6	a[0][0], a[0][1]	1	0	0xAB1FC143B8	b[0], b[1]
1	0	0xAB1FC143A6	a[0][2], a[0][3]	1	0	0xAB1FC143B8	b[2], b[3]
0	0			0	0		
0	0			0	0		
0	0			0	0		

	Address (Hex)	Tag	Index	
&a[0][0]	0x558FE0A1D330	0xAB1FC143A6	0x3	miss
&b[0]	0x558FE0A1DC30	0xAB1FC143B8	0x3	miss
&a[0][1]	0x558FE0A1D338	0xAB1FC143A6	0x3	hit
&b[1]	0x558FE0A1DC38	0xAB1FC143B8	0x3	hit
&a[0][2]	0x558FE0A1D340	0xAB1FC143A6	0x4	miss
&b[2]	0x558FE0A1DC40	0xAB1FC143B8	0x4	miss
&a[0][3]	0x558FE0A1D348	0xAB1FC143A6	0x4	hit
&b[3]	0x558FE0A1DC48	0xAB1FC143B8	0x4	hit
&a[0][4]	0x558FE0A1D350	0xAB1FC143A6	0x5	miss
&b[4]	0x558FE0A1DC50	0xAB1FC143B8	0x5	miss
&a[0][5]	0x558FE0A1D358	0xAB1FC143A6	0x5	hit
&b[5]	0x558FE0A1DC58	0xAB1FC143B8	0x5	hit
&a[0][6]	0x558FE0A1D360	0xAB1FC143A6	0x6	miss
&b[6]	0x558FE0A1DC60	0xAB1FC143B8	0x6	miss
&a[0][7]	0x558FE0A1D368	0xAB1FC143A6	0x6	hit
&b[7]	0x558FE0A1DC68	0xAB1FC143B8	0x6	hit
&a[0][8]	0x558FE0A1D370	0xAB1FC143A6	0x7	miss
&b[8]	0x558FE0A1DC70	0xAB1FC143B8	0x7	miss
&a[0][9]	0x558FE0A1D378	0xAB1FC143A6	0x7	hit
&b[9]	0x558FE0A1DC78	0xAB1FC143B8	0x7	hit

Taxonomy/reasons of cache misses

3Cs of misses

- Compulsory miss
 - Cold start miss. First-time access to a block
- Capacity miss
 - The working set size of an application is bigger than cache size
- Conflict miss
 - Required data replaced by block(s) mapping to the same set
 - Similar collision in hash

**How can programmer improve
memory performance?**

Data structures

Array of structures or structure of arrays

Array of objects			object of arrays												
average of each homework	<pre>struct grades { int id; double *homework; double average; };</pre>			<pre>struct grades { int *id; double **homework; double *average; };</pre>											
	ID	*homework	average	ID	*homework	average									
				<table><tr><td>ID</td><td>ID</td><td>ID</td></tr><tr><td>homework</td><td>homework</td><td>homework</td></tr><tr><td>average</td><td>average</td><td>average</td></tr></table>	ID	ID	ID	homework	homework	homework	average	average	average		
ID	ID	ID													
homework	homework	homework													
average	average	average													
	<pre>for(i=0;i<homework_items; i++) { gradesheet[total_number_students].homework[i] = 0.0; for(j=0;j<total_number_students;j++) gradesheet[total_number_students].homework[i] +=gradesheet[j].homework[i]; gradesheet[total_number_students].homework[i] /= (double)total_number_students; }</pre>			<pre>for(i = 0;i < homework_items; i++) { gradesheet.homework[i][total_number_students] = 0.0; for(j = 0; j <total_number_students;j++) { gradesheet.homework[i][total_number_students] += gradesheet.homework[i][j]; } gradesheet.homework[i][total_number_students] /= total_number_students; }</pre>											

Column-store or row-store

- If you're designing an in-memory database system, will you be using

RowId	Empld	Lastname	Firstname	Salary
1	10	Smith	Joe	40000
2	12	Jones	Mary	50000
3	11	Johnson	Cathy	44000
4	22	Jones	Bob	55000

- column-store — stores data tables column by column

10:001, 12:002, 11:003, 22:004;

Smith:001, Jones:002, Johnson:003, Jones:004;

Joe:001, Mary:002, Cathy:003, Bob:004;

40000:001, 50000:002, 44000:003, 55000:004;

if the most frequently used query looks like –
select Lastname, Firstname from table

- row-store — stores data tables row by row

001:10, Smith, Joe, 40000;

002:12, Jones, Mary, 50000;

003:11, Johnson, Cathy, 44000;

004:22, Jones, Bob, 55000;

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss

Loop interchange/fission/fusion

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

$O(n^2)$

Same

Same

Better

Complexity

Instruction Count?

Clock Rate

CPI

$O(n^2)$

Same

Same

Worse

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```



B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```



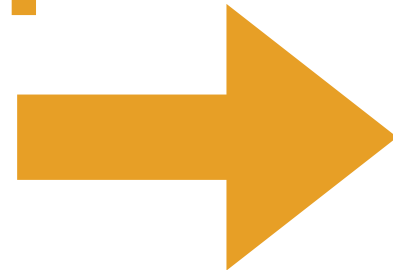
B

```
double a[8192], b[8192], c[8192], \  
       d[8192], e[8192];  
for(i = 0; i < 512; i++) {  
    e[i] = (a[i] * b[i] + c[i])/d[i];  
}
```

Loop fission

A

```
double a[8192], b[8192], c[8192], \  
       d[8192], e[8192];  
for(i = 0; i < 512; i++)  
    e[i] = a[i] * b[i] + c[i];  
for(i = 0; i < 512; i++)  
    e[i] /= d[i];
```



Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```



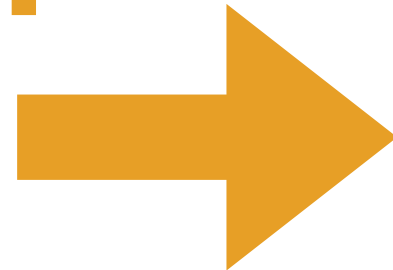
B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```

Loop fission

A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```



A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```

Loop fusion

B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```



Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity

Tiling/Blocking Algorithm

What is an M by N "2-D" array in C?

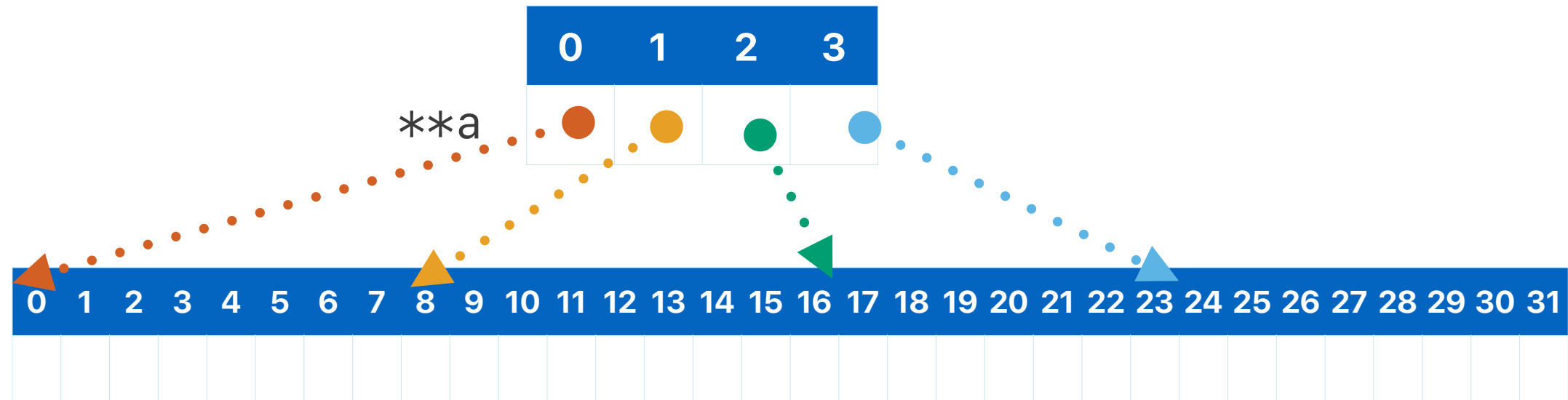
```
a = (double **)malloc(M*sizeof(double *));  
for(i = 0; i < N; i++)  
{  
    a[i] = (double *)malloc(N*sizeof(double));  
}
```

$a[i][j]$ is essentially $a[i*N+j]$

abstraction

	0	1	2	3	4	5	6	7
0								
1								
2								
3								

physical implementation



Case study: Matrix Multiplication

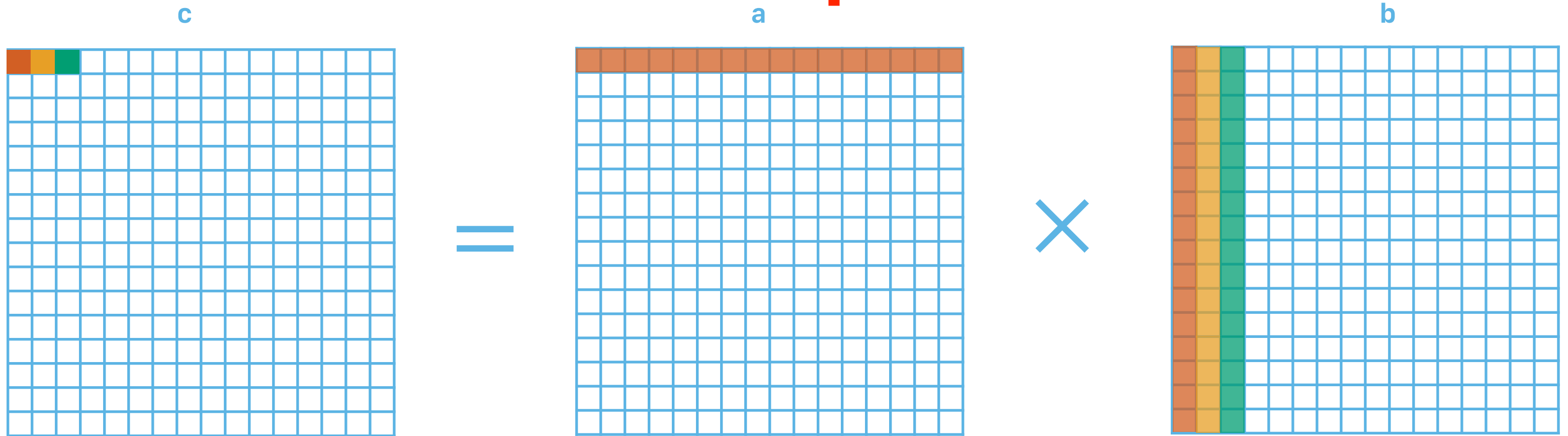
```
for(i = 0; i < M; i++) {  
    for(j = 0; j < N; j++) {  
        for(k = 0; k < K; k++) {  
            c[i][j] += a[i][k]*b[k][j];  
        }  
    }  
}
```

Algorithm class tells you it's $O(n^3)$

If $M=N=K=1024$, it takes about 1 sec

How long is it take when $M=N=K=2048$?

Matrix Multiplications



```
for(i = 0; i < M; i++) {  
    for(j = 0; j < N; j++) {  
        for(k = 0; k < K; k++) {  
            c[i][j] += a[i][k]*b[k][j];  
        }  
    }  
}
```

Matrix Multiplication — let's consider "b"

```
for(i = 0; i < M; i++) {  
  for(j = 0; j < N; j++) {  
    for(k = 0; k < K; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

- If the row dimension (N) of your matrix is 2048, each row element with the same column index is

$$2048 \times 8 = 16384 = 0x4000$$

away from each other

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] ...

Now, when we work on c[0][1]

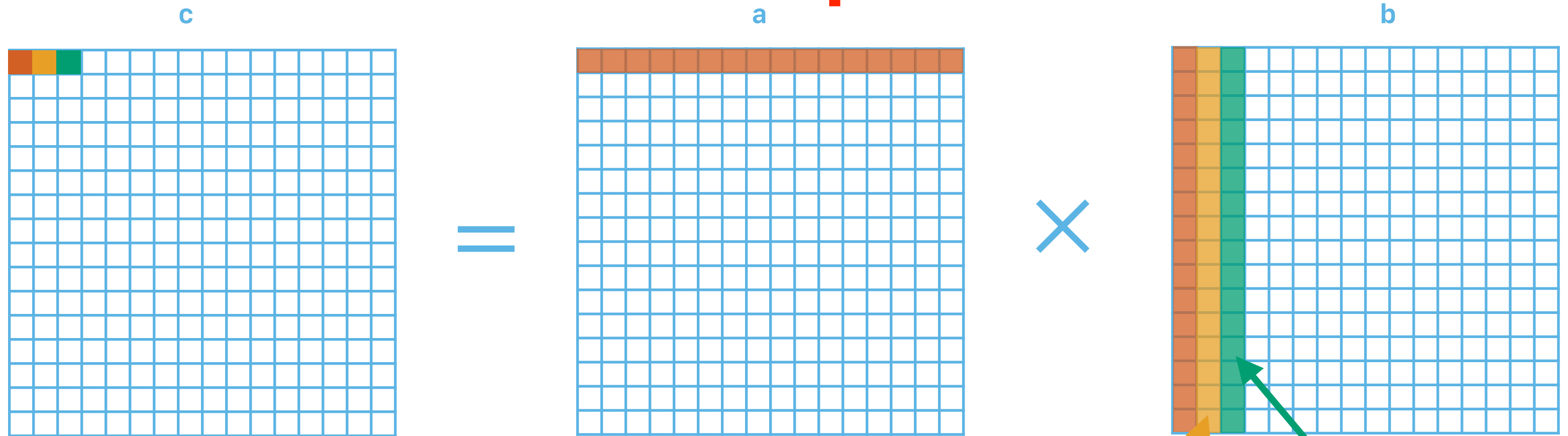
	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0



	Address	Tag	Index		
b[0][1]	0x20008	0x20	0x0	Conflict	Miss
b[1][1]	0x24008	0x24	0x0	Conflict	Miss
b[2][1]	0x28008	0x28	0x0	Conflict	Miss
b[3][1]	0x2C008	0x2C	0x0	Conflict	Miss
b[4][1]	0x30008	0x30	0x0	Conflict	Miss
b[5][1]	0x34008	0x34	0x0	Conflict	Miss
b[6][1]	0x38008	0x38	0x0	Conflict	Miss
b[7][1]	0x3C008	0x3C	0x0	Conflict	Miss
b[8][1]	0x40008	0x40	0x0	Conflict	Miss
b[9][1]	0x44008	0x44	0x0	Conflict	Miss
b[10][1]	0x48008	0x48	0x0	Conflict	Miss
b[11][1]	0x4C008	0x4C	0x0	Conflict	Miss
b[12][1]	0x50008	0x50	0x0	Conflict	Miss
b[13][1]	0x54008	0x54	0x0	Conflict	Miss
b[14][1]	0x58008	0x58	0x0	Conflict	Miss
b[15][1]	0x5C008	0x5C	0x0	Conflict	Miss
b[16][1]	0x60008	0x60	0x0	Conflict	Miss

Each set can store only 12 blocks! So
we will start to kick out b[0][0-7], b[1][0-7] ...

Matrix Multiplications

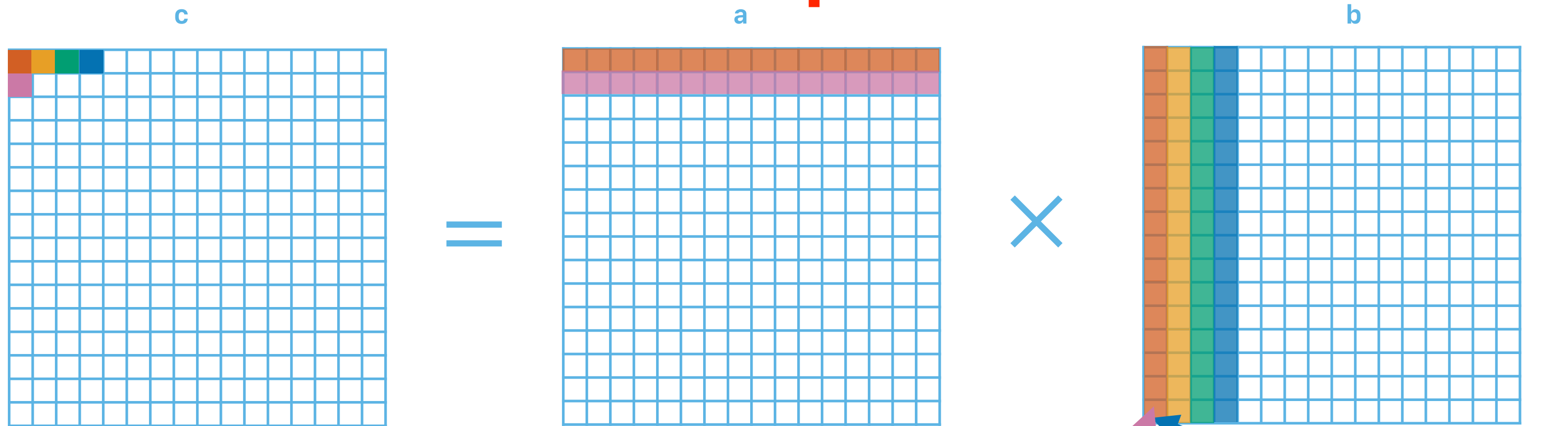


```
for(i = 0; i < M; i++) {  
  for(j = 0; j < N; j++) {  
    for(k = 0; k < K; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

These are
conflict misses
as we have
cached them
before

These are
conflict misses
as we have
cached them
before

Matrix Multiplications



- If each dimension of your matrix is 2048
 - Each row or column takes $2048 \times 8 \text{ Bytes} = 16 \text{ KB}$
 - The L1-\$ of intel Core i7 is 48 KB, 12-way, 64-byte blocked
 - You can only hold at most 3 rows or columns of each matrix!
 - You need the more columns when j increase!
- We will have capacity misses when we work on a new 1

We need to
fetch
everything
again —
capacity miss!

Unlikely to be
kept in the
cache

Ideas regarding reducing misses in matrix multiplications

- Reducing conflict misses — we need to reduce the length of a column that we visit within a period of time
- Reducing capacity misses — we need to reduce the length of a row that we visit within a period of time

Mathematical view of MM

$$\begin{aligned} c_{i,j} &= \sum_{k=0}^{k=N-1} a_{i,k} \times b_{k,j} = \sum_{k=0}^{k=\frac{N}{2}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{2}}^{k=N-1} a_{i,k} \times b_{k,j} \\ &= \sum_{k=0}^{k=\frac{N}{4}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{4}}^{k=\frac{N}{2}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{2}}^{k=\frac{3N}{4}-1} a_{i,k} \times b_{k,j} + \sum_{k=3\frac{N}{4}-1}^{k=N-1} a_{i,k} \times b_{k,j} \end{aligned}$$

Let's break up the multiplications and accumulations into something fits in the cache well

Matrix Multiplication — let's consider "b"

```
for(i = 0; i < M; i++) {  
  for(j = 0; j < N; j++) {  
    for(k = 0; k < K; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

- If the row dimension of your matrix is 2048, each row element with the same column index is

$$2048 \times 8 = 16384 = 0x4000$$

away from each other

If we stop at somewhere before 12 blocks, we should be fine!

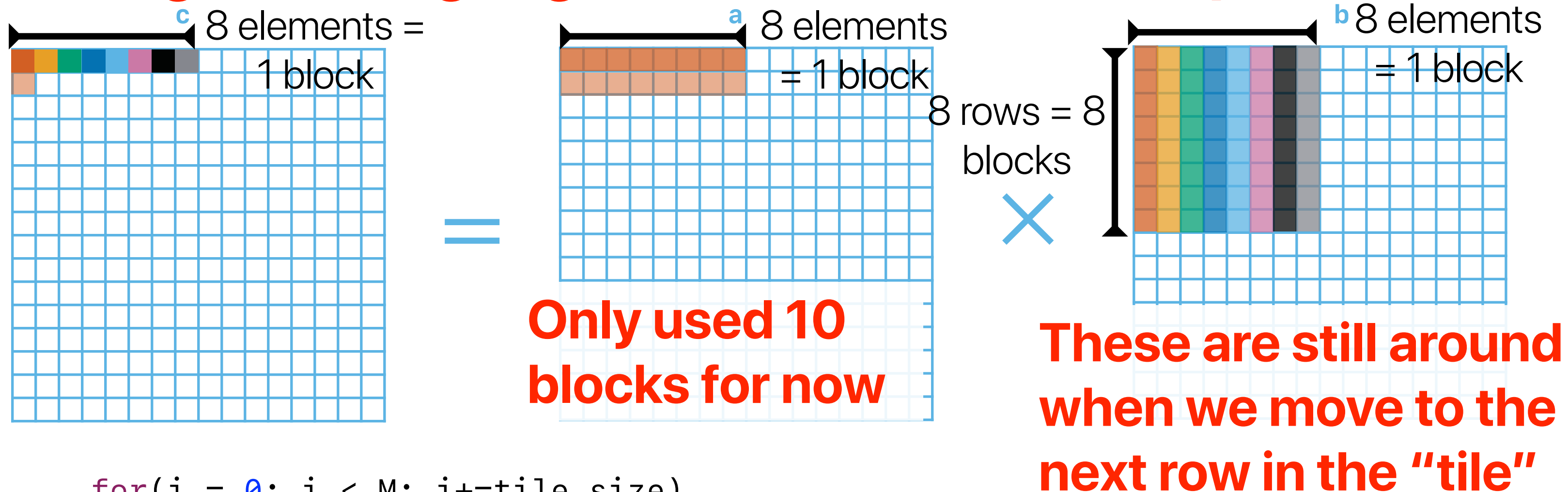
Since each block has 8 elements, let's break up in 8 for now

— 8 elements from a[i]

— 8 columns each covers 8 rows

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

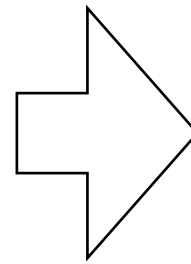
Tiling/Blocking Algorithm for Matrix Multiplications



```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < N; j+=tile_size)
        for(k = 0; k < K; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                    for(kk = k; kk < k+tile_size; kk++)
                        c[ii][jj] += a[ii][kk]*b[kk][jj];
```

Matrix Transpose

```
for(i = 0; i < M; i+=tile_size) {  
    for(j = 0; j < N; j+=tile_size) {  
        for(k = 0; k < K; k+=tile_size) {  
            for(ii = i; ii < i+tile_size; ii++)  
                for(jj = j; jj < j+tile_size; jj++)  
                    for(kk = k; kk < k+tile_size; kk++)  
                        c[ii][jj] += a[ii][kk]*b[kk][jj];  
        }  
    }  
}
```



```
// Transpose matrix b into b_t  
for(i = 0; i < ARRAY_SIZE; i+=(ARRAY_SIZE/n)) {  
    for(j = 0; j < ARRAY_SIZE; j+=(ARRAY_SIZE/n)) {  
        b_t[i][j] += b[j][i];  
    }  
}  
  
for(i = 0; i < M; i+=tile_size) {  
    for(j = 0; j < N; j+=tile_size) {  
        for(k = 0; k < K; k+=tile_size) {  
            for(ii = i; ii < i+tile_size; ii++)  
                for(jj = j; jj < j+tile_size; jj++)  
                    for(kk = k; kk < k+tile_size; kk++)  
                        // Compute on b_t  
                        c[ii][jj] += a[ii][kk]*b_t[jj][kk];  
        }  
    }  
}
```

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity
- Blocking/tiling — capacity miss, conflict miss